



Concurrency Control

Agenda

- ▶ **Concurrency Control**
- ▶ **Lock-Based Protocol**
- ▶ **Deadlocks & Starvation**



Concurrency Control

Concurrency Control

- ▶ **Concurrency Control** in a **Database Management System (DBMS)** is the process of managing simultaneous operations (transactions) on the database without allowing them to interfere with each other — ensuring **data consistency, isolation, and correctness**.

Goals of CC

1. **Maintain Data Consistency** – Ensure the database remains in a consistent state before and after transactions.
2. **Preserve Isolation** – One transaction should not see the intermediate results of another.
3. **Improve Throughput** – Allow maximum parallelism without conflicts.
4. **Prevent Anomalies** – Avoid issues like deadlocks, lost updates, and dirty reads.



Lock-Based Protocol

Lock based protocol

- **The most common method to implement concurrency control is using a lock**
- **A lock in DBMS controls concurrent access, allowing only one transaction to use a data item at a time.**
- **Lock Based Protocols in DBMS ensure that a transaction cannot read or write data until it gets the necessary lock.**

How do the Locks work

- **These protocols prevent concurrency issues by allowing only one transaction to access a specific data item at a time.**
- **A transaction must acquire a read lock or write lock on a data item before performing any read or write operations on it.**

Types of Locks – Shared Lock

- **Shared Lock is also known as Read-only lock.**
- **As the name suggests it can be shared between transactions because while holding this lock the transaction does not have the permission to update data on the data item.**
- **S-lock is requested using lock-S instruction.**

Types of Locks – Exclusive Lock

- **Data item can be both read as well as written. This is Exclusive and cannot be held simultaneously on the same data item.**
- **X-lock is requested using lock-X instruction.**

Locks

- **Every transaction must request to the concurrency-control manager.**
- **The transaction can proceed with the operation only after the concurrency-control manager grants the lock to the transaction.**

Lock Compatibility

- **A transaction can acquire a lock on a data item only if the requested lock is compatible with existing locks held by other transactions.**
- **Shared Locks (S): Multiple transactions can hold shared locks on the same data item simultaneously.**

Lock Compatibility

- **Exclusive Lock (X):** If a transaction holds an exclusive lock on a data item, no other transaction can hold any type of lock on that item.
- If a requested lock is not compatible, the requesting transaction must wait until all incompatible locks are released by other transactions.
- Once the incompatible locks are released, the requested lock is granted.

Lock Compatibility

	S	X
S	✓	X
X	X	X

Example

Two transactions, T1 and T2, update the same data item A (Without Locks).

Step	T1	T2
1	Read A = 100	
2		Read A = 100
3	$A = A + 50 \rightarrow 150$	
4	Write A = 150	
5		$A = A - 30 \rightarrow 70$
6		Write A = 70

Example

Two transactions, T1 and T2, update the same data item A (With Locks).

T1:

LOCK-X(A)

READ(A) = 100

A = A + 50

WRITE(A) = 150

UNLOCK(A)

T2:

WAIT (A locked by T1)

LOCK-X(A)

READ(A) = 150

A = A - 30

WRITE(A) = 120

UNLOCK(A)

Two Phase Locking Protocol (2PL)

- A transaction is said to follow the Two-Phase Locking protocol if Locking and Unlocking can be done in two phases :
- **Growing Phase:** New locks on data items may be acquired but none can be released.
- **Shrinking Phase:** Existing locks may be released but no new locks can be acquired.



Deadlocks & Starvation

Deadlock

- **A deadlock occurs in a database system when two or more transactions are waiting for each other to release locks, and none of them can proceed. It's a state of permanent blocking.**

DEADLOCKS

- ▶ Unfortunately, delayed locking can also lead to an undesirable situation.
- ▶ Consider the partial schedule as shown here for T_3 and T_4 .
- ▶ Since T_3 is holding an exclusive-mode lock on B and T_4 is requesting a shared-mode lock on B , T_4 is waiting for T_3 to unlock B .
- ▶ Similarly, since T_4 is holding a shared-mode lock on A and T_3 is requesting an exclusive mode lock on A , T_3 is waiting for T_4 to unlock A .
- ▶ Thus, we have arrived at a state where neither of these transactions can ever proceed with its normal execution. This situation is called *deadlock*.

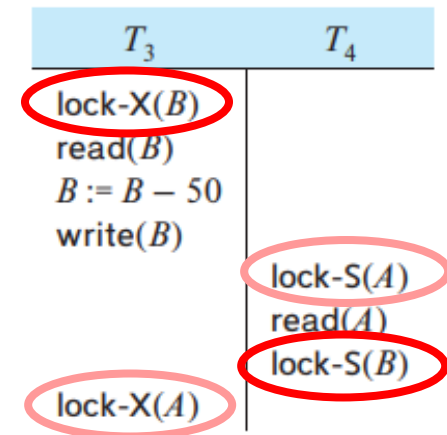
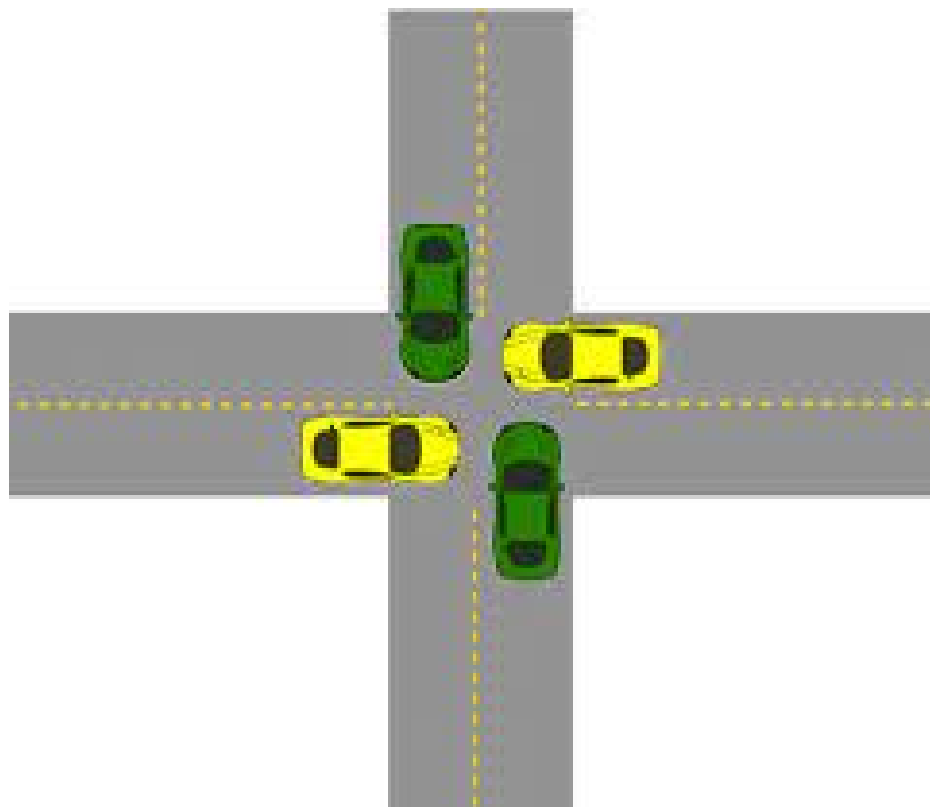


Figure 18.7 Schedule 2.

DEADLOCK (TRAFFIC EXAMPLE)



Starvation

- **Starvation occurs when a transaction is continuously denied access to a resource (like a lock) it needs to proceed — even though it's not deadlocked.**
- **It happens when other transactions keep getting priority, and the waiting transaction never gets a chance to execute.**

Starvation example

Example Scenario

Suppose three transactions are trying to access the same data item A:

pgsql

T1 → needs an Exclusive Lock (X) on A

T2 → needs a Shared Lock (S) on A

T3 → needs a Shared Lock (S) on A

Starvation example

Time	Transaction	Lock Type
t1	T2 requests S(A)	Granted
t2	T3 requests S(A)	Granted (shared allowed)
t3	T1 requests X(A)	Waiting (because others hold shared locks)
t4	T2 releases S(A)	—
t5	T3 again requests S(A)	Granted again
t6	T1 still waiting...	✗ Starvation continues

AVOIDING STARVATION

- ▶ We can avoid starvation of transactions by granting locks in the following manner:
- ▶ When a transaction T_i requests a lock on a data item Q in a particular mode M , the concurrency-control manager grants the lock provided that:
 1. *There is no other transaction holding a lock on Q in a mode that conflicts with M*
 2. *There is no other transaction that is waiting for a lock on Q and that made its lock request before T_i*
- ▶ Thus, a lock request will never get blocked by a lock request that is made later.



THANK YOU